

TRABAJO PRÁCTICO N ° 4

Listas e Iteradores

Departamento de Ciencias e Ingeniería de la Computación - U.N.S.

Ejercicio 1:

Con el objetivo de implementar el TDALista con una estructura doblemente enlazada con centinelas:

- Utilice la interface PositionList<E> dada por la cátedra;
- Implemente la clase DNodo<E> que implemente la interface Position<E>;
- Utilice y lance las excepciones pertinentes;
- Implemente un iterador para la lista programada.
- Ejecute el tester JUnit provisto por la cátedra para verificar la correctitud de su TDALista.

Ejercicio 2:

Agregue un método a la lista programada en el ejercicio 1 tal que reciba dos elementos, *e1* y *e2*, y modifique la lista receptora del mensaje de la siguiente manera:

- Deberá agregar a *e1* como segundo elemento de la lista;
- Deberá agregar a *e2* como ante-último elemento de la lista;

Considera casos especiales: ¿Qué sucede si la lista está vacía?, ¿Qué sucede si la lista tiene un elemento?. Recuerde que tiene total acceso a la estructura.

Ejercicio 3:

Utilizando el iterador programado en el TDALista:

- Dada una PositionList<E> *I* y un elemento genérico *e1*, escriba un método tal que determine si *e1* se encuentra en la lista *I*. Compare los elementos por equivalencia.
- Dada una PositionList<E> *I* y un elemento genérico *e1*, escriba un método tal que retorne la cantidad de veces que *e1* se encuentra en la lista *I*. Compare los elementos por equivalencia.
- Dada una PositionList<E> *I*, un elemento genérico *x* y un entero *n*, retorne verdadero si y sólo si el elemento *x* se encuentra en la lista *I* al menos *n* veces. Compare los elementos por identidad.

Ejercicio 4:

Escriba un método tal que dada una PositionList<E> *I*, retorne una nueva lista que tenga los elementos de *I* en el mismo orden pero repetidos. Por ejemplo:

Si *I*=[a, b, c, d] el método deberá retornar *INueva*=[a, a, b, b, c, c, d, d]

Para resolver este ejercicio deberá utilizar la sentencia for-each.

Ejercicio 5:

Escriba un método tal que dadas dos `PositionList<Character>` *l1* y *l2* elimine de *l2* todos los elementos que también están en *l1*. Este método debe retornar un iterable con todos los elementos eliminados.

Ejercicio 6:

- a) Escriba un método que tenga como entrada dos listas genéricas *L1* y *L2* y retorne una nueva lista resultante de la intercalación de ambas.
- b) Dadas dos listas de enteros ordenadas *L1* y *L2*, se desea obtener una tercera lista ordenada producto de la intercalación de *L1* y *L2*. En la lista resultante no debe haber elementos repetidos.

OBS: en ambos casos tener en cuenta que las listas pueden tener distintas longitudes.

Ejercicio 7:

Implemente un método `eliminar(L1,L2)`, que modifique la lista *L1* de la siguiente manera: primero deberá eliminar de la misma todas las apariciones de los elementos contenidos en *L2*, luego deberá insertar al final de la misma todos los elementos de *L2* pero en orden inverso al que aparecen en *L2*. Puede considerar que *L2* no tiene elementos repetidos. No se debe modificar el estado interno de la lista *L2*.

TIPS:

- Antes de comenzar a desarrollar este trabajo práctico repase el diagrama de clases completo para visualizar todas las clases e interfaces que intervienen en la implementación del `TDALista`.
- Para programar el iterador utilice la estrategia de iterar sobre la estructura (no sobre una copia).
- Para que todo funcione correctamente utilice las interfaces `Iterable` e `Iterator` provistas por Java (`java.util`), y las interfaces y clases provistas por la cátedra en los paquetes en las que fueron declaradas.
- Cada vez que resuelva un ejercicio considere si el ejercicio debe resolverse dentro de la estructura (con acceso a la estructura de nodos) o fuera de la misma (como cliente usando solamente las operaciones del TDA).
- Es muy útil esbozar dibujos de las listas para ver cómo se deben modificar los enlaces ante determinadas situaciones.